

OTHELLO

ΑΝΤΩΝΙΟΥ ΔΗΜΗΤΡΙΟΣ ΑΜ:3230011
ΖΕΡΒΑΣ ΓΕΩΡΓΙΟΣ ΑΜ:3230330

1. Εισαγωγή

Στα πλαίσια της 1ης εργασίας του μαθήματος, αναπτύξαμε λογισμικό σε C++ που επιτρέπει στον χρήστη να παίξει το επιτραπέζιο παιχνίδι Othello εναντίον του υπολογιστή. Ο πυρήνας της τεχνητής νοημοσύνης του υπολογιστή βασίζεται στον αλγόριθμο **MiniMax** με κλάδεμα **Alpha-Beta** (Alpha-Beta Pruning), καθώς και στη Ευρετική Συνάρτηση που θα παρουσιάσουμε παρακάτω.

2. Τρόπος Χρήσης του Προγράμματος

Η εφαρμογή εκτελείται μέσω γραμμής εντολών, όπου κατά την εκκίνηση ο χρήστης καλείται να ορίσει τη σειρά του παιχνιδιού (επιλέγοντας '0' για τον υπολογιστή ή '1' για τον ίδιο) καθώς και το επιθυμητό βάθος αναζήτησης του αλγορίθμου, το οποίο καθορίζει πόσο βαθιά θα τρέξει ο αλγόριθμος minimax. Κατά τη διάρκεια της παρτίδας, η κατάσταση του ταμπλό απεικονίζεται με χαρακτήρες, ενώ ο χρήστης εισάγει τις κινήσεις του δίνοντας τις αντίστοιχες συντεταγμένες (γραμμή και στήλη), με το σύστημα να ελέγχει αυτόματα την εγκυρότητά τους και να ζητά διόρθωση σε περίπτωση σφάλματος. Η διαδικασία ολοκληρώνεται όταν γεμίσει το ταμπλό ή όταν δεν υπάρχουν πλέον διαθέσιμες κινήσεις για κανέναν παίκτη, οπότε και αναδεικνύεται ο νικητής βάσει του συνολικού αριθμού πούλιων.

3. Δυνατότητες του Λογισμικού

Το λογισμικό παρέχει τις εξής λειτουργίες:

- **Παιχνίδι Man vs Pc:** Υλοποίηση πλήρους παρτίδας Othello.
- **Έλεγχος Εγκυρότητας Κινήσεων:** Το σύστημα αναγνωρίζει αυτόματα ποιες κινήσεις είναι νόμιμες σύμφωνα με τους κανόνες (περικύκλωση αντιπάλου).
- **Αυτόματο "Πάσο":** Αν ένας παίκτης δεν έχει κινήσεις, το σύστημα το αναγνωρίζει, εμφανίζει μήνυμα και δίνει τη σειρά στον αντίπαλο, όπως ορίζουν οι κανόνες.
- **Βάθος:** Ορίζει πόσο θα 'σκεφτεί' το σύστημα για να επιλέξει κίνηση(δηλαδή πόσες αναδρομικές κλήσεις θα κάνει ο μινιμαξ-οσο πιο πολλές τόσο πιο πιθανό να φτάσει στην

καλύτερη κίνηση, όμως ταυτόχρονα πολλές αναδρομικές κλήσεις σημαίνει και μεγαλύτερος χρόνος)

- **Ευρετική Συνάρτηση:** Αξιολόγηση κατάστασης βάσει θέσης και πλήθους πούλιων.

4. Αρχιτεκτονική Συστήματος

Ο κώδικας μας για καλύτερη ευκολία και ανάγνωση, μοιράστηκε σε τρία αρχεία:

1. **othello.h (Header):** Περιέχει τη δήλωση της κλάσης othello, καθώς και τις συναρτήσεις που χρειάζονται για το πρόγραμμα. Επίσης ορίζεται ο πίνακας `array[ BOARD_SIZE][ BOARD_SIZE]` -έχει αρχικοποιηθεί στο 8 το `BOARD_SIZE`.
2. **othello.cpp (Implements):** Περιέχει την υλοποίηση των μεθόδων

1. `play():` Εκτελεί μια κίνηση και αναστρέφει (flips) τα πούλια.
2. `get_all_valid_moves_for_player():` Επιστρέφει λίστα με όλες τις πιθανές κινήσεις.
3. `minimax():` Ο αναδρομικός αλγόριθμος της TN.
4. `evaluate():` Η ευρετική συνάρτηση αξιολόγησης.

Αυτές οι 4 είναι που συντονίζουν όλο το παιχνίδι και πανω σε αυτές λειτουργούν και οι υπόλοιπες.

3. **main.cpp (Main Game):** Διαχειρίζεται τη ροή του παιχνιδιού (Συγκεκριμένα: εναλλαγή γύρων, έλεγχος τερματισμού, είσοδος/έξοδος).

5. Μέθοδοι Τεχνητής Νοημοσύνης

Για τη λήψη αποφάσεων από τον υπολογιστή χρησιμοποιήθηκαν οι εξής μέθοδοι, σύμφωνα με την εκφώνηση:

5.1 Αλγόριθμος MiniMax

Ο αλγόριθμος MiniMax κατασκευάζει το δέντρο του παιχνιδιού μέχρι το επιλεγμένο βάθος (`depth`) και συγκεκριμένα:

- Ο υπολογιστής λειτουργεί ως **Maximizer**, προσπαθώντας να μεγιστοποιήσει την τιμή της ευρετικής συνάρτησης.
- Ο χρήστης θεωρείται **Minimizer**, προσπαθώντας να ελαχιστοποιήσει το κέρδος του υπολογιστή.

```

/*minimax*/
int othelo::minimax(int current_depth,int depth, bool isMax,char maximizing_player,int alpha,int beta)
{
    /*opponent player*/
    char minimizing_player;
    if (maximizing_player=='0')
        minimizing_player = '1';
    else
        minimizing_player = '0';
    /*current player*/
    char current_player;
    if(isMax)
        current_player = maximizing_player;
    else
        current_player = minimizing_player;

    std::vector<std::pair<int, int>> valid_moves = this->get_all_valid_moves_for_player(current_player);

    if (valid_moves.empty())/*o trelos palkto den exei epkiri kinthi,opote thelame eite pass,eite game over*/
    {
        /*elagosome an e antipalon opori na kinithel*/
        if (this->get_all_valid_moves_for_player(minimizing_player).empty()) {
            return this->evaluate(maximizing_player);
        }

        return this->minimax(current_depth, depth, !isMax, maximizing_player,alpha,beta);/*pass*/
    }

```

5.2 Κλάδεμα Άλφα-Βήτα (Alpha-Beta Pruning)

Για τη βελτιστοποίηση της ταχύτητας, ενσωματώθηκε το κλάδεμα α-β.(Pruning)

- Διατηρούνται δύο τιμές, **alpha** (η best τιμή για τον Maximizer) και **beta** (η best τιμή για τον Minimizer).
- Αν σε κάποιο κλαδί βρεθεί κίνηση χειρότερη από αυτή που έχει ήδη εξασφαλίσει ο αντίπαλος (**beta <= alpha**), το κλαδί αυτό "κόβεται" (pruning) και δεν εξετάζεται περαιτέρω.

```

if(isMax)
{
    int max_evaluation = -10000;
    for(const auto& move : valid_moves)
    {
        othelo new_board(*this);
        new_board.play(move.first, move.second);
        int eval = new_board.minimax(current_depth + 1, depth, false, maximizing_player,alpha,beta);
        max_evaluation = std::max(eval,max_evaluation);

        alpha = std::max(alpha, max_evaluation);
        if (alpha >= beta) {
            break;
        }
    }

    return max_evaluation;
}
else
{
    int min_evaluation = 10000;
    for(const auto& move : valid_moves)
    {
        othelo new_board(*this);
        new_board.play(move.first, move.second);
        int eval = new_board.minimax(current_depth +1,depth,true,maximizing_player,alpha,beta);
        min_evaluation = std::min(eval,min_evaluation);
        beta = std::min(beta, min_evaluation);
    }
}

```

5.3 Ευρετική Συνάρτηση (Heuristic Evaluation)

Η συνάρτηση `evaluate(char maximizing_player)` είναι υπεύθυνη για την ανάθεση της τιμής `score` σε κάθε κατάσταση του ταμπλό. Η τιμή αυτή χρησιμοποιείται για σύγκριση πιθανων καταστάσεων και επιλογής της πιο συμφέρουσας για τον υπολογιστή. Η υλοποίηση βασίζεται σε έναν γραμμικό συνδυασμο 2 ευρετικών, ευρετική θέσεων και διαφοράς πούλιων.

1. **Ευρετική Θέσεων - Πίνακας Βαρών (Position Weights):** Κάθε κελί του ταμπλό έχει μια σταθερή αξία.
Συγκεκριμένα: Οι **γωνίες** έχουν τιμή **+120**, καθώς είναι τα πιο σημαντικά σημεία (δεν αναστρέφονται). Κελιά γύρω από τις γωνίες έχουν τιμή **-20** ή **-40**, καθώς η καταληψη τους συχνά παραχωρεί τη γωνία στον αντίπαλο. Οι πλευρές έχουν θετική αξία (+5 ή +20).
2. **Διαφορά Πούλιων:** Συνυπολογίζεται η διαφορά (Πούλια PC - Πούλια Παικτη). Η τελική αξιολόγηση προκύπτει από τον τύπο: $\text{final_score} = (\text{total_score} * 1.0) + (\text{Heuristic_Count} * 0.8)$

```
/* get_all_valid_moves_for_player*/
std::vector<std::pair<int, int>> get_all_valid_moves_for_player(char player)
{
    std::vector<std::pair<int, int>> moves; /* moves = lista py perioroi ta zygaria ton agkiron klironom*/
    int directions[8][2] = {{-1,-1}, {-1,0}, {-1,1}, {0,-1}, {0,1}, {1,-1}, {1,0}, {1,1}}; /* 8 katyethnias kateinontai apo pouw arisi */
    int dx, dy;
    for(int i=0; i<BOARD_SIZE; i++)
        for(int j=0; j<BOARD_SIZE; j++)
            if(array[i][j]=='-')
            {
                /*gia authn thn thesi, elagxoume kai tis 8 katyethnias*/
                for(int k=0; k<8; k++)
                {
                    dx = directions[k][0];
                    dy = directions[k][1];

                    if (is_valid_direction[i, j, dx, dy, player]) /*anethike agkiri klirsh */
                    {
                        moves.push_back({i,j}); /*epitheti to stinelo stn lista kai ankwnei to negetho kata i*/
                        break;
                    }
                }
            }
    return moves;
}
```

```

int othello :: evaluate(char maximizing_player)/*euretikh diaforas pouliwn kai barh*/
{
    static const int weights[BOARD_SIZE][BOARD_SIZE] = {
        {120, -20, 20, 5, 5, 20, -20, 120},
        {-20, -40, -5, -5, -5, -5, -40, -20},
        {20, -5, 15, 3, 3, 15, -5, 20},
        {5, -5, 3, 3, 3, 3, -5, 5},
        {5, -5, 3, 3, 3, 3, -5, 5},
        {20, -5, 15, 3, 3, 15, -5, 20},
        {-20, -40, -5, -5, -5, -5, -40, -20},
        {120, -20, 20, 5, 5, 20, -20, 120}
    };

    int total_score = 0;
    int count_pc=0;
    int count_opponent=0;
    char opponent_player;

    if (maximizing_player == 'O') {
        opponent_player = 'I';
    } else {
        opponent_player = 'O';
    }
}

```

Συγκεκριμένα:

Για να κρίνει ο υπολογιστής πόσο καλή είναι μια κατάσταση, συνδυάζει δύο κριτήρια: τη **Βαθμολογία θέσης** (πόσο καλά είναι τοποθετημένα τα πούλια, π.χ. αν έχουμε πιάσει γωνίες) και τη **Διαφορά Πούλιων** (ποιος έχει τα περισσότερα). Το έξυπνο σημείο είναι ότι δίνει διαφορετική σημασία στο καθένα χρησιμοποιώντας **συντελεστές βάρους**: πολλαπλασιάζει τη βαθμολογία θέσης με **1.0** και τη διαφορά πούλιων με **0.8**. Αυτό γίνεται επειδή η **στρατηγική θέση** είναι πιο σημαντική από τον απλό αριθμό των πούλιων, οπότε ο αλγόριθμος προτιμά να πιάσει μια ασφαλή γωνία παρά να έχει απλώς πολλά πούλια που μπορεί να του τα φάνε αργότερα.

```

for (int i = 0; i < BOARD_SIZE; i++)
{
    for (int j = 0; j < BOARD_SIZE; j++)
    {
        if (array[i][j] == maximizing_player)
        {
            total_score += weights[i][j];/*An vroyme poulli toy "maximizing_player" (AI), prosthetoyme to varos count_pc++;/*metavliith pou krataei posa poulia exei o pc sto desk*/
        }

        else if (array[i][j] == opponent_player)
        {
            total_score -= weights[i][j];/* An vroyme to pouli toy antipaleoy, afairowme to varos count_opponent++;/*metavliiti pou krataei posa poulia exei o player sto desk*/
        }
    }
}
double Heuristic_count = double(count_pc-count_opponent);

const double W_Position = 1.0;
const double W_Count = 0.8;
double final_score = (total_score*W_Position) + (Heuristic_count*W_Count);/*athroisaa ginomenwn twv 2 euretikwn gia to kalitero ap
return (int)final_score;

```

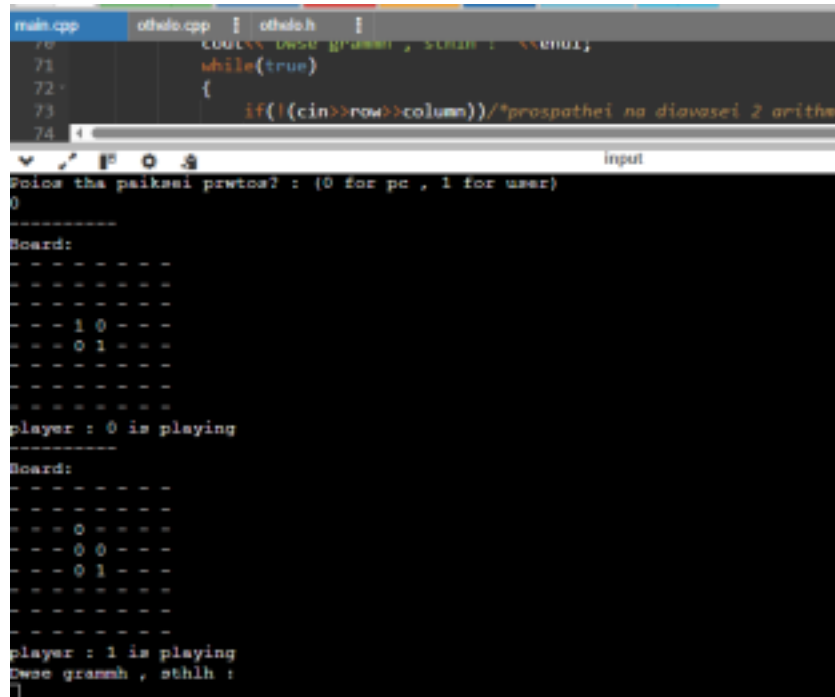
6. Παραδείγματα Εκτέλεσης

Παράδειγμα 1: Έναρξη παιχνιδιού και επιλογή ποιός θα παίξει πρώτος καθώς και του depth.(PC vs Paiktis)

```
int main() {
    othelo o1;
    char player = '1';
    char ai = '0';
    int depth ;
    char choice;
    int choice_i;
    int choice_j;
    cout<<"Poios tha paiksei prwtos? : (0 for pc , 1 for user)"<<endl;
    cin>>choice;
    o1.set_player(choice);

    cout<<"Dialekse to vathos(depth) toy minimax: "<<endl;
    cin>>depth;
```

Παράδειγμα 2: Κίνηση Υπολογιστή(depth)



```
main.cpp | othelo.cpp | othelo.h |
70      cout<<"Dialekse to vathos(depth) toy minimax: "<<endl;
71      while(true)
72      {
73          if(!((cin>>row)>>column))/*prosopatheï na diavasei 2 arithm
74          {
              input
Poios tha paiksei prwtos? : (0 for pc , 1 for user)
0
-----
Board:
-----
-- 1 0 --
-- 0 1 --
-- 0 0 --
-----
player : 0 is playing
-----
Board:
-----
-- 0 0 --
-- 0 0 --
-- 0 1 --
-----
player : 1 is playing
Dialekse to vathos(depth) toy minimax:
0
```

Παράδειγμα 3: Error: Λανθασμένη Κίνηση Παίκτη

```

Polos tha paiksei prwtos? : (0 for pc , 1 for user)
0
-----
Board:
- - - - -
- - - - -
- - - - -
- - 1 0 - -
- - 0 1 - -
- - - - -
- - - - -
- - - - -

player : 0 is playing
-----
Board:
- - - - -
- - - - -
- - 0 - - -
- - 0 0 - -
- - 0 1 - -
- - - - -
- - - - -
- - - - -

player : 1 is playing
Dwse grammh , stlh :
2
3
Error,Dwse swatew sintetagenes!

```

Παράδειγμα 4: Κανονική Ροή Παιχνιδιού(End of Game)

```

player : 1 is playing
Dwse grammh , stlh :
7
1
6
3
Kinhsh egkyrh
-----
Board:
0 - 0 - - - -
- 0 0 - - - -
0 1 0 0 0 0 -
1 1 0 1 0 - -
- - 1 1 1 - -
0 1 0 1 0 0 -
1 - - 1 - - 0 -
- - - - - 0

player : 0 is playing
-----
Board:
0 - 0 - - - -
- 0 0 - - - -
0 1 0 0 0 0 -
1 1 0 1 0 - -
- - 1 1 1 - -
0 1 0 1 0 0 -
0 - - 1 - - 0 -
0 - - - - - 0

player : 1 is playing
Dwse grammh , stlh :
Killed

...Program finished with exit code 9
Press ENTER to exit console.

```

Παράδειγμα 5-Αποτέλεσμα ολόκληρου παιχνιδιού(νικητής : υπολογιστής)

```
player : 0 is playing
-----
Board:
0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 0 1 0 0 1 0
0 0 1 0 0 1 1 0
1 0 0 0 0 1 1 0
0 0 0 0 0 0 - 0
player : 1 is playing
0 paikths 1 den exei egkyrh kinshh kai paramenei h seira
-----
Board:
0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 0 1 0 0 1 0
0 0 1 0 0 1 1 0
1 0 0 0 0 1 1 0
0 0 0 0 0 0 - 0
player : 0 is playing
-----
Board:
0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 0 1 0 0 0 0
0 0 1 0 0 1 0 0
1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
player : 1 is playing
Game is overPc won
PS C:\Users\giorz\Desktop\3230811_3230330\code> |
```

Συμπεράσματα :

- 1)Κόστος αναζήτησης : ο αλγοριθμος μινιμαξ εξετάζει το δέντρο εκθετικά ως προς το βάθος N Πολυπλοκότητα $O(b^N)$ -χειρότερη περίπτωση.
- 2)Το πριόνισμα a-b pruning,μειώνει σημαντικά τον χρόνο εκτέλεσης,ιδανικά και σε $O(b^{(N/2)})$,επιτρέποντας τον μινιμαξ να φτάνει σε μεγαλύτερο depth σε λιγότερο χρόνο.Ωστόσο η πολυπλοκότητα παραμένει εκθετική,και συνηθως βρισκεται αναμεσα σε $O(b^N)$ και $O(b^{(N/2)})$,καθώς μπορεί το πριόνισμα να χρησιμοποιηθεί ελάχιστες φορές ή και καθόλου,οπότε μπορεί να μην συμβάλει στην βελτιστοποίηση της απόδοσης.Παράδειγμα, για $N \geq 5$ ο χρόνος αυξάνεται σημαντικά-όμως η κίνηση του υπολογιστή θα είναι πολύ πιο 'δυνατή'.